

DDTA - Working Metamodel - Macro Requirement

NON CANONICAL WORKING DRAFT

This file defines the general document metamodel. ThreatForge-specific observations are excluded and belong to the case-study material.

1. Scope

The metamodel describes the knowledge of a governed project before deciding its concrete Doc-as-Code serialization and before deciding how a software engine validates or assists authoring.

The governed product remains documentation-as-code: versionable, diffable, reviewable files (for example YAML registries and Markdown bodies). A software tool may apply and verify the metamodel, but it is not the subject described by the project's documents.

Layer separation

1. **Domain metamodel** - meaning of documents, entities and relations.
2. **Doc-as-Code representation** - how that meaning is serialized in files.
3. **Engine** - how tools load, validate, correlate, project and assist those files.

2. Principles already fixed

Principle 1 - Progressive specialization and semantic containment

Governed project knowledge is represented as a hierarchy of progressively specialized governed documents. A Macro Requirement establishes a macro-level subject and purpose; Decisions narrow that subject through explicit choices; Functional Requirements translate those choices into independently verifiable obligations; Specialized Requirements add domain-specific constraints to one Functional Requirement.

Hierarchical parentage represents semantic containment and remains distinct from non-hierarchical traceability, evidence and provenance relations.

Principle 2 - Identity independent from textual encoding

Every governed entity has a stable identity independent from the textual encoding of that identity. Hierarchy is established by explicit semantic relationships. Composite identifier formats may mirror ancestry but must not be the authoritative source of topology.

3. Progressive disclosure by competence

The hierarchy increases both specificity and the amount of vertical competence required from the reader.

- **Macro Requirement** - understandable by stakeholders who understand the project domain, including non-software stakeholders.
- **Decision** - requires more knowledge of the solution space and project choices.
- **Functional Requirement** - precise, testable project behaviour or outcome.
- **Specialized Requirement** - vertical competence such as governance, security, quality or another project-specific discipline.
- **Analysis models and methodology-specific records** - specialist analytical concepts introduced only when needed.

Executive-map criterion

Reading only the **title and Intent** of all Macro Requirements of a governed project must produce a recognizable summary of:

- what the project is trying to realize;
- what general value it produces;

- why its main blocks of work exist.

This criterion applies to the **governed project**, not to the tool used to govern it.

4. Domain vocabulary versus metamodel vocabulary

The metamodel must not ban technical words merely because they are technical.

A term belongs in an MR when it is part of the **application domain** and is reasonably understandable to the stakeholders of that domain. For example, a cryptography project may naturally use security terminology; a medical imaging project may naturally use imaging terminology; a facial-recognition access project may naturally use terms such as facial recognition, biometric data or ML model.

What the MR must not be forced to contain are technical concepts introduced **only because the documentation model or an analysis method expects them**, such as:

- component inventories;
- data-flow decompositions;
- trust boundaries;
- protocol details;
- attack-surface classifications;
- STRIDE/LINDDUN categories;
- method-specific threat or privacy taxonomies.

Those concepts normally emerge later, when Decisions, Requirements, a project/system representation or methodology-specific analysis require them.

5. Macro Requirement definition

A Macro Requirement identifies one major project concern or macro-capability. It explains why that concern matters, who is affected, and the general result the project seeks at that level.

It is the root document type of the governed-document hierarchy.

Fundamental question

Which major project concern or result are we governing, why does it matter, and who is involved?

6. Candidate information model

Concept	State	Purpose
id	required	stable identity, history and traceability
title	required	concise name understandable within the project domain
lifecycle	required	current/historical state of the document
intent	required	concise explanation of why the MR exists and why it matters
context	required	minimum background needed to understand the concern
stakeholders	required	people/groups that benefit, use, govern or are affected
objective	candidate required	macro result sought; not a list of atomic obligations
scope	open semantic decision	boundary of the macro concern; exact form still under study

Concept	State	Purpose
assumptions	optional	facts accepted as true that materially affect the whole MR
constraints	optional	limits that materially constrain the whole MR
non_goals	open semantic decision	plausible expectations deliberately outside the MR, if useful

Concision

A Macro Requirement must be effective rather than verbose. It should not become a checklist of implementation rules.

- **title:** short noun phrase or short descriptive title;
- **intent:** normally 1-2 sentences;
- **context:** only the background needed to interpret the MR;
- **stakeholders:** macro stakeholder roles, not a technical inventory;
- **objective:** normally a small number of high-level results;
- **assumptions/constraints:** only if they affect the whole branch below the MR.

7. Decisions already closed

7.1 Objective is not a list of must

The MR does not own atomic, independently verifiable obligations. Those belong to Requirements. **Objective** describes a macro result, not a long normative checklist.

7.2 Stakeholders are semantically required

The MR must identify the stakeholder groups relevant to its macro concern. Their names may be technical when that is normal domain language, but they should be meaningful to the project community rather than internal implementation labels.

7.3 Assumptions and Constraints are distinct

- **Assumption:** something currently treated as true for the branch of work.
- **Constraint:** a limit within which the branch must operate.

Both are optional and belong in the MR only when they materially influence the whole branch below it.

7.4 Parent and children

`MacroRequirement.parent = null.`

A Macro Requirement may own 0..* Decisions. The child collection is derived from `Decision.parent -> MacroRequirement`; it is not a second topology authority.

7.5 Cross-MR dependency

A Macro Requirement may have a non-hierarchical `dependsOn -> MacroRequirement [0..*]` relation when another macro concern is a genuine dependency. The exact Doc-as-Code serialization is deferred.

7.6 No Acceptance Criteria at MR level

The MR does not contain atomic acceptance criteria. Evidence of satisfaction emerges from its descendant Requirements and their verification evidence.

7.7 Progressive technicality

The MR should be understandable by stakeholders competent in the **project domain** without requiring them to understand implementation decomposition or analysis methodology. More specialized technical knowledge is introduced as documentation descends.

7.8 Technical and analytical decomposition normally emerges later

The MR must not be structurally required to enumerate components, services, APIs, data flows, trust boundaries, attack surfaces or threat categories. Those normally emerge at lower documentation or analysis levels.

This does **not** prohibit domain-specific terminology in an MR.

7.9 MR prose must be stable with respect to design progress

A Macro Requirement describes a durable project concern, not the current progress of solution design. Statements such as **not yet decided**, **currently evaluating**, **will be chosen later** or equivalent design-status prose normally belong to working notes, Decision lifecycle/status or another planning artifact rather than to the MR body.

Use the **temporal-stability test**: if a sentence becomes obsolete merely because a lower-level Decision is taken, while the macro project concern remains unchanged, the sentence should normally not be part of the MR.

This rule does not forbid time-dependent **domain facts**. For example, an authorization may expire or a biometric enrollment may have a lifecycle because those are properties of the application domain. The rule targets temporary authoring/design state, not legitimate temporal semantics of the project.

8. MR invariants

1. **Root hierarchy** - MR has no parent.
2. **Stable identity** - identity is distinct from its textual encoding.
3. **Single macro concern** - the MR expresses one coherent macro responsibility.
4. **Child containment** - descendants progressively narrow the MR rather than introduce unrelated scope.
5. **No implementation choice** - choices of solution belong to Decision unless they are intrinsic to the project domain itself.
6. **No requirement-level obligation** - atomic testable obligations belong to Requirements.
7. **Broad audience within the domain** - title and Intent can be understood by non-software stakeholders who understand the project domain.
8. **Project-map usefulness** - titles + Intent of all MR summarize the project and its major work blocks.
9. **Explicit dependencies** - cross-MR dependencies are not hidden as pseudo-hierarchy.
10. **Analysis enabling, method agnostic** - MR provides project knowledge but does not prescribe a threat-analysis paradigm.
11. **Architecture resilience** - changing a lower-level architecture choice should not normally force the MR to be rewritten if the macro project concern has not changed.
12. **Temporal stability** - taking or revising a lower-level Decision should not make MR prose obsolete unless the macro project concern itself changes.

9. Semantic decisions still open

These points must be tested with concrete examples before the MR model is frozen:

9.1 Intent versus Objective

We need examples to determine whether both concepts add distinct information or whether they create unnecessary duplication.

Candidate distinction:

- **Intent** - why this MR exists / why it matters;
- **Objective** - what macro result we want to obtain.

This distinction is **not yet frozen**.

9.2 Meaning and shape of Scope

We need examples to determine what **scope** should actually express and whether **included**, **excluded** and **non_goals** are all necessary or partly redundant.

The goal is not to create three places that repeat the same boundary in different words.

9.3 When to split or merge Macro Requirements

MR boundaries must not be decided from wording similarity alone. This is especially important when refactoring an existing governed corpus: two rewritten MRs can sound similar because both were described through the same downstream consumer, while their descendant Decisions still govern different project concerns. Conversely, historical separation is not sufficient evidence to keep two MRs if it only mirrors an implementation split.

Before splitting or merging candidate MRs, test the boundary with these questions:

1. **Independent value** - does each candidate express a project-level result that remains meaningful without describing the other?
2. **Decision-family coherence** - where an existing corpus already has Decisions, do the descendants of each candidate form distinct and internally coherent families of choices?
3. **Solution-vocabulary removal** - does the distinction survive after removing names of current components, schemas, tools, analysis representations and other chosen mechanisms?
4. **Dependency versus containment** - can the relation be expressed more accurately as **dependsOn**, rather than by merging concerns or introducing false hierarchy?
5. **Stakeholder explanation** - can a stakeholder competent in the project domain explain the difference using title + Intent without knowing the implementation?
6. **Merge stress test** - would a merged MR need to explain substantially different families of Decisions or different kinds of project value?
7. **Split stress test** - is the separation justified by project concerns, rather than merely by architectural layers, subsystems or workflow stages?

These are currently **boundary-test heuristics**, not yet frozen deterministic invariants. They must be exercised on multiple project examples.

Refactoring rule for existing documentation When an MR already owns a body of Decisions, reconstruct its macro concern from the semantic responsibility of that Decision family before rewriting the MR. A rewritten MR should explain why those Decisions belong together without merely summarizing their current solution vocabulary.

10. Observation deferred to the future analysis metamodel

Concrete MR examples show that different concerns inside the same governed project may later benefit from different analysis paradigms. An ML-based capability may justify AI/ML-oriented threat analysis, while another concern in the same project may justify privacy-oriented analysis or a different specialist method.

This observation does **not** change the MR contract and must not introduce method-specific fields into Macro Requirements. The project hierarchy describes project concerns; analysis methodology is a later concern.

The future analysis metamodel should therefore test whether it can:

1. select an analysis method for an explicit analysis scope or set of governed subjects rather than assigning one methodology to an MR by construction;
2. allow different methods to analyze different, overlapping or complementary parts of one project;
3. preserve method-specific interpretation without contaminating project knowledge with methodology vocabulary;
4. converge findings, documentation gaps, proposed requirements and other feedback into one governed project/documentation lifecycle.

These are **deferred design questions**, not frozen analysis-model decisions. They are recorded here so that the MR model does not accidentally preclude multi-method project analysis.

11. Doc-as-Code mapping deliberately deferred

The following remain representation questions, not domain-model decisions:

- where each MR concept lives between registry and Markdown body;
- whether `parent = null` is serialized or implicit;
- concrete representation of stakeholders, assumptions and constraints;
- prose versus list representation of Objective;
- concrete representation of `dependsOn`;
- final Markdown heading names;
- deterministic checks that can enforce the semantic contract without pretending to validate human meaning.

STOP: Decision is not modeled in this document yet.